R

RUST·EZE
WORKSHOP MANAGER

MEMORIA DE PROYECTO

RUST·EZE WORKSHOP MANAGER

Plataforma de gestión integral para jefes de taller mecánico — centralización de equipo, clientes, vehículos y reparaciones en un único entorno.

VERSIÓN	STACK	AÑO
1.0	NODE · EXPRESS · POSTGRESQL	2026

DOCUMENTO TÉCNICO

ÍNDICE

Tabla de Contenidos

Estructura completa del documento de memoria del proyecto Rust-eze Workshop Manager.

01 Introducción 3

02 Objetivos del Proyecto 3

03 Tecnologías Utilizadas 4

04 Arquitectura y Flujo de Funcionamiento 5

05 Diseño del Sistema 7

06 Desarrollo del Proyecto 9

07 Implementaciones Técnicas Destacadas 10

08 Problemas Encontrados y Soluciones 11

09 Conclusiones 12

10 Mejoras Futuras 12

11 Anexos Técnicos 13

Resumen ejecutivo — Rust-eze es una plataforma web backend desarrollada con Node.js, Express 5, Sequelize y PostgreSQL, contenerizada con Docker, que permite a los talleres mecánicos digitalizar su operación diaria mediante un sistema de roles diferenciado y dashboards personalizados en tiempo real.

01

Introducción

Rust-eze Workshop Manager es una plataforma web de gestión interna diseñada específicamente para jefes de taller mecánico. El problema que resuelve es concreto: los talleres tradicionales gestionan clientes, vehículos, reparaciones y equipos de mecánicos de forma fragmentada, usando papel, hojas de cálculo o herramientas genéricas que no se adaptan a su flujo real de trabajo.

Rust-eze centraliza toda esta información en un único entorno digital, permitiendo al jefe de taller tener visibilidad completa de su operación en tiempo real: qué reparaciones están activas, qué mecánicos las llevan, qué clientes tiene asignados y cuál es el estado económico de cada intervención.

El enfoque del proyecto no se limita al resultado funcional, sino también a la calidad arquitectónica del código, aplicando separación de responsabilidades, patrones de diseño reconocibles en la industria y entorno de desarrollo reproducible mediante contenedores Docker.

Problema

Gestión fragmentada en papel, Excel y herramientas genéricas no adaptadas al flujo de un taller.

Solución

Entorno digital único con visibilidad total del taller, equipo y operación en tiempo real.

02

Objetivos

2.1 Objetivo general

Desarrollar una aplicación web backend robusta y escalable que permita a los jefes de taller gestionar de forma centralizada su equipo, clientes, vehículos y reparaciones, con control de acceso por roles.

2.1 Objetivos específicos

- Implementar autenticación segura con gestión de sesiones mediante express-session
- Diseñar un sistema de roles diferenciado: Admin (jefe de taller) y Mecánico
- Desarrollar un dashboard personalizado por rol con métricas operativas relevantes.
- Modelar y relacionar correctamente las entidades del dominio: talleres, usuarios, vehículos, reparaciones, tareas, reservas y facturas.
- Contenerizar el entorno completo con Docker para garantizar reproducibilidad.
- Renderizar vistas dinámicas con Pug como motor de plantillas.

03

Tecnologías Utilizadas

El stack tecnológico se ha seleccionado priorizando estabilidad, ecosistema maduro y reproducibilidad del entorno de desarrollo entre máquinas distintas.

Tecnología	Rol en el proyecto
Node.js	Entorno de ejecución del servidor
Express 5	Framework HTTP y gestión de rutas
PostgreSQL 16	Base de datos relacional
Sequelize	ORM para modelado y consultas
Pug	Motor de plantillas para vistas dinámicas
express-session	Gestión de sesiones de usuario
bcryptjs	Hash seguro de contraseñas
Docker / Docker Compose	Contenerización del entorno completo
pgAdmin 4	Administración visual de la base de datos
nodemon	Recarga automática en desarrollo

Criterio de selección — Cada tecnología cubre una responsabilidad concreta sin solapamientos. La combinación Express + Sequelize + Pug forma la base de una arquitectura SSR clásica, mientras Docker garantiza que el entorno funcione idénticamente en cualquier m áqui na.

04

Arquitectura y Flujo de Funcionamiento

4.1 Patrón arquitectónico: MVC + Service Layer

El proyecto implementa una arquitectura en capas que extiende el patrón MVC clásico añadiendo una capa de servicios explícita. Esta decisión separa claramente cuatro responsabilidades:

```
Rutas → Middlewares → Controladores → Servicios → Modelos → Base de datos
```

La ventaja práctica es que l lógica de negocio vive en los servicios y puede reutilizarse desde distintos controladores sin duplicar código. Por ejemplo: `dashboardService.getAdminDashboard()` podría llamarse tanto desde una vista Pug como desde un futuro endpoint de API REST sin modificar nada. La estructura de carpetas quedará reflejada en los anexos técnicos.

4.2 Flujo de una petición autenticada

1. Usuario accede a `GET /dashboard`
2. `isLoggedIn` verifica `req.session.user`
3. Si no hay sesión → redirect `/login`
4. Si hay sesión → `getDashboard(req, res)`
5. Controlador lee `req.session.user.rol`
6. Según el rol llama a `getAdminDashboard()` o `getEmployerDashboard()`
7. El service ejecuta las queries a través de Sequelize
8. Los datos se pasan a `res.render('dashboard-admin', { data })`
9. Pug compila la vista con los datos y responde al navegador

4.3 Flujo de autenticación

```
POST /login
   ↓
checkCredentials (middleware)
 → busca usuario por email
 → compara contraseña con bcryptjs
 → si correcto: guarda req.user
   ↓
login (controlador)
 → crea req.session.user con { id, rol, id_taller, nombre, email }
   ↓
redirect /dashboard
```

4.4 Sistema de roles

El sistema distingue tres roles definidos en la tabla `roles` de la base de datos:

ID	Rol	Dashboard	Acceso
1	Admin	`dashboard-admin`	Visión completa del taller, clientes y reparaciones activas
2	Mecánico	`dashboard-employer`	Vista de su taller y las reparaciones que le afectan
3	Cliente	`dashboard-client`	Vista de los clientes de su taller

El controlador del dashboard actúa como enrutador de vistas según el rol:

```
if (rol === 1) → getAdminDashboard(id)
if (rol === 2) → getEmployerDashboard(id)
```

Diagrama de capas

```
┌──────────────────────────────────────────────────────────┐ │ PRESENTACIÓN Pug Templates
· Chart.js │ ├──────────────────────────────────────────────────────────┤ │ RUTAS /dashboard
/login /repairs │ ├──────────────────────────────────────────────────────────┤ │ MIDDLEWARES
isLoggedIn · injectUserToViews│
├──────────────────────────────────────────────────────────┤ │ CONTROLADORES Orquestan
request/response │ ├──────────────────────────────────────────────────────────┤ │
SERVICIOSLógica de negocio reusable │
├──────────────────────────────────────────────────────────┤ │ MODELOS Sequelize ORM │
├──────────────────────────────────────────────────────────┤ │ BASE DE DATOS PostgreSQL
16 │ └──────────────────────────────────────────────────────────┘
```

05

Diseño del Sistema

5.1 Estructura del proyecto

```
src/
├── config/          → Conexión y sincronización con la BD
├── controller/
│   └── views/       → Controladores de vistas Pug
├── middlewares/     → Auth, validación de sesión, inyección de usuario
├── models/          → Definición de entidades Sequelize
├── routes/
│   └── views/       → Rutas organizadas por dominio
├── services/        → Lógica de negocio y consultas complejas
├── utils/           → Utilidades y manejo de errores
└── views/           → Plantillas Pug
    ├── layout.pug
    ├── dashboard-admin.pug
    ├── dashboard-employer.pug
    ├── dashboard-client.pug
    ├── auth.pug
    ├── repair.pug
    ├── repairDetail.pug
    ├── reservation.pug
    └── errors/

db/
└── scripts/         →SQL de schema y datos iniciales
public/              →CSS estático
```

5.2 Modelo de datos

El dominio se articula alrededor de ocho entidades con las siguientes relaciones:

```
ROLES ──< USERS >── TALLERES │ ├──< CARS >──< REPAIRS >──< TASKS │ └ INVOICES
└──< RESERVATIONS >── TALLERES
```

La relación más importante para el dashboard es la cadena `TALLERES → USERS → CARS → REPAIRS`, ya que no existe una FK directa entre talleres y reparaciones. Las reparaciones se filtran por taller a través de los usuarios propietarios de los vehículos, lo que requiere JOINs anidados gestionados mediante `include` encadenados de Sequelize con `required: true`.

5.3 Decisiones de diseño relevantes

¿Por qué express-session en lugar de JWT?
Para una aplicación web con vistas renderizadas en servidor, express-session es la opción más apropiada. El estado de sesión vive en el servidor, lo que permite invalidarla de forma inmediata. JWT tiene sentido cuando existe un frontend desacoplado o clientes móviles que consumen una API. Si en el futuro Rust-eze expone una API REST pública, la migración a JWT sería directa.

¿Por qué bcryptjs en lugar de bcrypt nativo?
bcrypt nativo compila binarios C++ que pueden comportarse de forma diferente según el sistema operativo del contenedor Docker, generando incompatibilidades al comparar hashes. bcrypt js es implementación pura de JavaScript, comportándose de forma idéntica en cualquier entorno.

¿Por qué Sequelize en lugar de SQL directo?
El ORM permite definir las relaciones entre entidades una sola vez en models/index.js y explotarlas mediante include en cualquier consulta. Para un proyecto con ocho tablas relacionadas como Rust-eze, esto reduce significativamente el riesgo de errores en los JOINs y hace el código más legible.

06

Desarrollo del Proyecto

Fase 1 — Análisis y diseño del dominio

Definición del propósito de Rust-eze: centralizar la gestión operativa de talleres mecánicos. Identificación de los actores (jefe de taller, mecánico, cliente) y sus necesidades. Diseño del modelo entidad-relación con ocho tablas y sus relaciones. Selección del stack tecnológico.

Fase 2 — Configuración del entorno

Inicialización del proyecto Node.js con estructura modular. Configuración de Express con middlewares esenciales. Montaje de Docker Compose con tres servicios: backend, PostgreSQL y pgAdmin. Resolución de problemas de comunicación entre contenedores y tiempos de inicialización de la base de datos mediante `healthcheck`.

Fase 3 — Modelado de datos

Implementación del schema SQL con las ocho tablas y sus relaciones de integridad referencial. Definición de los modelos Sequelize y sus asociaciones en `models/index.js`. Creación del seed de datos con usuarios, vehículos, reparaciones, tareas e invoices con datos coherentes entre sí. Resolución de problemas de extensión de archivos en `docker-entrypoint-initdb.d` — PostgreSQL solo ejecuta automáticamente archivos `.sql`.

Fase 4 — Autenticación y gestión de sesiones

Implementación del registro de usuarios con validación de datos, verificación de email único y hash de contraseña con `bcry ptj s`. Implementación del login con verificación de credenciales. Sistema de sesiones con `express-session`. Middleware `isLoggedIn` para proteger rutas. Middleware `injectUserToViews` para disponibilizar el usuario en todas las plantillas Pug. Resolución de un problema crítico: el hook `beforeCreate` del modelo de Sequelize estaba hasheando la contraseña una segunda vez después de que el middleware ya la había hasheado, haciendo imposible el login.

Fase 5 — Dashboard y vistas

Implementación del dashboard diferenciado rol. Desarrollo del servicio `getAdminDashboard` con consultas encadenadas para obtener clientes y reparaciones activas filtradas por taller, navegando la cadena TALLERES → USERS → CARS → REPAIRS Resolución del error `EagerLoadingError` causado por incluir modelos sin asociación definida, y del error `Car is null` causado por usar LEFT JOIN donde se necesitaba INNER JOIN (`required: true`). Creación de las vistas Pug para admin, mecánico y cliente con tablas de reparaciones activas, gráficas con Chart.js y listado de clientes.

07

Implementaciones Técnicas Destacadas

Consulta de reparaciones activas por taller

La ausencia de FK directa entre reparaciones y talleres requirió una consulta con tres niveles de `include` anidados en Sequelize, usando `required: true` en cada nivel para forzar INNER JOIN y excluir reparaciones de otros talleres.

Sistema de sesiones

La sesión almacena `{ id, rol, id_taller, nombre, email }` del usuario autenticado. Este objeto está disponible en todos los controladores como `req.session.user` y en todas las vistas Pug como `user` mediante el middleware `injectUserToViews`.

Dashboard con Chart.js

El dashboard de administración incluye tres gráficas generadas en cliente a partir de datos inyectados desde el servidor mediante `!{JSON.stringify(data)}` en Pug:

- Distribución de marcas en reparación(pie)
- Coste por reparación(barras)
- Ratio clientes vs reparaciones activas(donut)

Inicialización automática de datos

Los scripts SQL del schema y del seed se ejecutan automáticamente al arrancar el contenedor PostgreSQL por primera vez, gracias al directorio `docker-entrypoint-initdb.d`. Esto garantiza que cualquier desarrollador que clone el repositorio tenga el entorno completamente funcional con un solo `docker compose up --build`.

08

Problemas Encontrados y Soluciones

Problema	Causa	Solución
Scripts SQL ignorados al arrancar Docker	Extensión `.db` no reconocida por PostgreSQL	Renombrar a `.sql`
`EagerLoadingError: Repairs is not associated`	Asociaciones no definidas en `models/index.js`	Centralizar todas las asociaciones en el archivo de í ndi ce
`Cannot read properties of null (reading 'brand')`	Sequelize devolvía null con LEFT JOIN al filtrar por taller	Añadir `required: true` en cada nivel del include
Login siempre fallaba tras el registro	El hook `beforeCreate` del modelo hasheaba la contraseña por segunda vez	Eliminar el hook, dejando el hash únicamente en el m i ddl ew are
Incompatibilidad de hashes bcrypt en Docker	bcrypt nativo genera binarios dependientes del SO del contenedor	Migrar a `bcry ptj s` (implementación pura JS)
Sesión vacía tras login correcto	`isLoggedIn` usaba `req.session.user` mientras el controlador guardaba en otro objeto	Unificar el sistema en `express-session` eliminando los restos de JWT
Permiso denegado al borrar `.database/data`	La carpeta fue creada por el proceso postgres del contenedor con permisos root	Usar `sudo rm -rf` o `sudo chown` para recuperar los permisos

09

Conclusiones

Rust-eze ha permitido aplicar de forma integrada los conceptos fundamentales del desarrollo backend moderno: arquitectura en capas, autenticación con sesiones, ORM con relaciones complejas, contenerización y renderizado de vistas dinámicas.

El aspecto más valioso del proceso ha sido enfrentarse a problemas reales que no aparecen en los tutoriales: incompatibilidades de librerías en Docker, doble hashing de contraseñas por hooks del ORM, JOINs que devuelven null en lugar de excluir filas, o archivos ignorados por el motor de inicialización de PostgreSQL. Resolverlos ha consolidado una comprensión más profunda del stack que cualquier ejercicio sintético

La arquitectura resultante es extensible :añadir nuevos roles, nuevas vistas o una API REST son operaciones que no requieren reescribir código existente gracias a la separación de responsabilidades aplicada desde el principio.

10

Mejoras Futuras

API REST
Exponer los endpoints existentes para permitir el consumo desde un f rontend React o una app móvil.

Recuperación de contraseña
Envío de email con enlace de reseteo seguro y caducidad temporal.

Notificaciones en tiempo real
WebSockets para alertar al jefe de taller cuando una reparación cambia de estado.

Módulo de informes
Exportación de datos a PDF o Excel por rango de fechas.

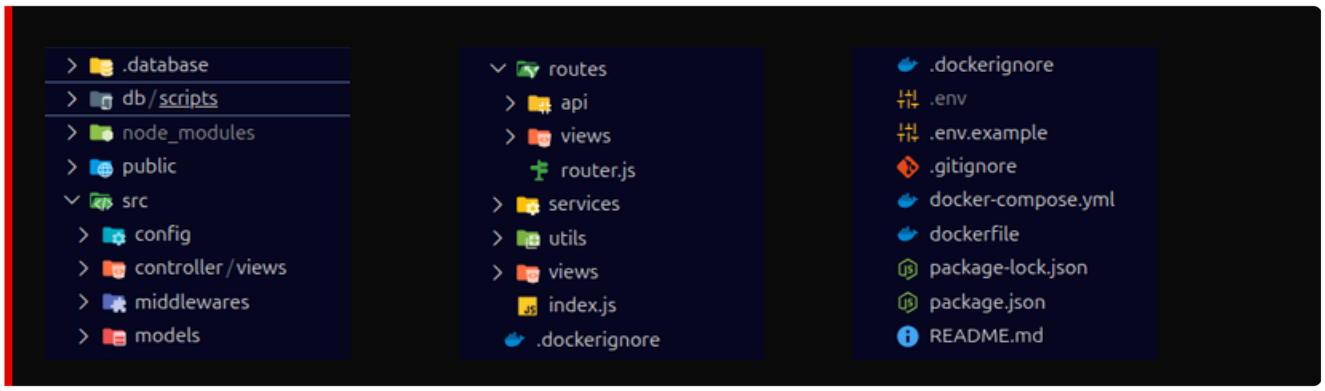
Multitenancy
Soporte para que un administrador gestione múltiples talleres desde una sola cuenta.

Tests automatizados
Cobertura de los servicios críticos con Jest e integración continua.

11

Anexos Técnicos

A. Estructura de carpetas



B. Comandos de arranque

```
# Primera vez o tras limpiar datos
sudo rm -rf .database/
docker compose up --build

# Arranque normal
docker compose up
```

C. Variables de entorno requeridas

```
DB_HOST=db
DB_PORT=5432
DB_U S E R=. . .
DB_PASSWORD=...
DB_NAME=rusteze_db
SESSION_SECRET=...
PORT= 3 0 0 0
H OS T= loc alh ost
```

RUST·EZE

WORKSHOP MANAGER · MEMORIA TÉCNICA · 2026